



TURMA INTERMEDIÁRIA
SEMANA 1
FENWICK TREE

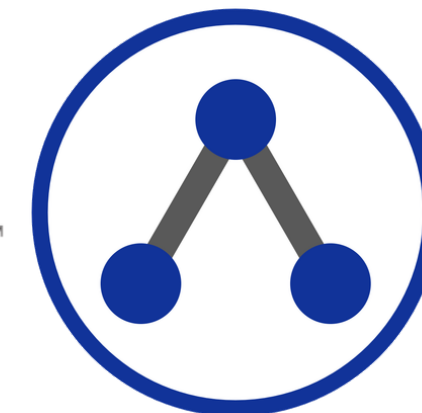
GABRIEL CAMPELO

João Pessoa, 2025

Motivação: Range Sum Query

Vetor $A[0..N-1]$ com duas operações:

- $\text{update}(i, x) : A[i] += x$
- $\text{query}(l, r) : A[l] + A[l+1] + \dots + A[r]$



CSES Problem Set

Dynamic Range Sum Queries

TASK | STATISTICS

Time limit: 1.00 s Memory limit: 512 MB

Given an array of n integers, your task is to process q queries of the following types:

1. update the value at position k to u
2. what is the sum of values in range $[a, b]$?

Abordagens Ingênuas

ABORDAGEM	UPDATE	QUERY	MEMÓRIA
Array Direto	$O(1)$	$O(N)$	$O(N)$
Prefix Sum	$O(N)$	$O(1)$	$O(N)$

Fenwick Tree

UPDATE	QUERY	MEMÓRIA
$O(\log N)$	$O(\log N)$	$O(N)$

A árvore de Fenwick também é chamada de Árvore Binária Indexada (BIT). Foi descrita pela primeira vez em um artigo intitulado "Uma nova estrutura de dados para tabelas de frequência acumulada" (Peter M. Fenwick, 1994)

Definições

- Vamos definir uma função de grupo **f** tal que **$f(x, y) = x * y$**
- Note que **'*'** é a notação infixa de **f**
- Agora, **query(l, r)** consiste em calcular **$A[l] * A[l + 1] * \dots * A[r]$**
- Para o nosso problema, **'*' = '+'**

DEFINIÇÃO 1.1. *Seja G um conjunto não vazio munido de uma operação binária*

$$* : G \times G \rightarrow G$$

$$(a, b) \mapsto a * b$$

*Dizemos que o par $(G, *)$ é um grupo se as seguintes condições são satisfeitas:*

*i) (**Associatividade**) Para quaisquer $a, b, c \in A$, tem-se que*

$$(a * b) * c = a * (b * c).$$

*ii) (**Existência de elemento neutro**) Existe $e \in G$ tal que $e * a = a * e = a$, qualquer que seja $a \in G$.*

*iii) (**Existência de elemento inverso**) Para todo $a \in G$, existe $a' \in G$ tal que $a * a' = a' * a = e$.*

Exemplo

O operador '+' é válido

- $(X + Y) + Z = X + (Y + Z)$ **(OK)**
- $0 + X = X + 0 = X$ **(OK)**
- $X + (-X) = (-X) + X = 0$ **(OK)**

O operador '^' também é válido

- $(X \wedge Y) \wedge Z = X \wedge (Y \wedge Z)$ **(OK)**
- $0 \wedge X = X \wedge 0 = 0$ **(OK)**
- $X \wedge X = X \wedge X = X$ **(OK)**

Fenwick Tree

A árvore de Fenwick será apenas um vetor de **N** posições, **T[0...N-1]**, em que cada elemento é aplicação da função **f** sobre um intervalo **[g(i), i]** do vetor **A** tal que **0 <= g(i) <= i**

$$T_i = \sum_{j=g(i)}^i A_j$$

Fenwick Tree

O bom funcionamento da BIT mora na escolha da função **g**

Contudo, já podemos definir a estrutura geral do algoritmo

```

1  int query(int r) {
2      int ret = 0;
3      while (r >= 0) {
4          ret += t[r];
5          r = g(r) - 1;
6      }
7      return ret;
8  }
10 void update(int i, int add) {
11     for (int j = 0; j < N; j++) {
12         if (g(j) <= i <= j) {
13             t[j] += add;
14         }
15     }
16 }

```

Nosso sonho é que os laços “while (r >= 0)” e “for (int j = 0; j < N; j++)” rodem em $O(\log N)$

Note que $\text{query}(l, r) = \text{query}(r) - \text{query}(l - 1)$

Escolha da função g

Se $g(i) = i$ então voltamos para o problema do **array direto**. Ainda, se $g(i) = 0$, estamos criando um vetor de **soma de prefixos**

A escolha ideal consiste em $g(i) = i \& (i + 1)$

Em suma, verificamos se o bit menos significativo é igual a **'1'** e, em caso afirmativo, flipamos o maior sufixo de **'1's'**

$$g(11) = g(1011_2) = 1000_2 = 8$$

$$g(12) = g(1100_2) = 1100_2 = 12$$

$$g(13) = g(1101_2) = 1100_2 = 12$$

$$g(14) = g(1110_2) = 1110_2 = 14$$

$$g(15) = g(1111_2) = 0000_2 = 0$$

$$g(11) = g(1011_2) = 1000_2 = 8$$

$$g(12) = g(1100_2) = 1100_2 = 12$$

$$g(13) = g(1101_2) = 1100_2 = 12$$

$$g(14) = g(1110_2) = 1110_2 = 14$$

$$g(15) = g(1111_2) = 0000_2 = 0$$

Queries resolvidas,
agora precisamos
lidar com os updates

Lidando com updates

Precisamos atualizar a nossa “árvore”. Para isso, é preciso atualizar $T[j]$ para todos j 's tais que $g(j) \leq i \leq j$

Felizmente, é possível definir um operador $h(j) = j \mid (j + 1)$ que faz esse caminho partindo de $T[i]$

$$10 = 0001010_2$$

$$h(10) = 11 = 0001011_2$$

$$h(11) = 15 = 0001111_2$$

$$h(15) = 31 = 0011111_2$$

$$h(31) = 63 = 0111111_2$$

$$\vdots$$

$$10 = 0001010_2$$

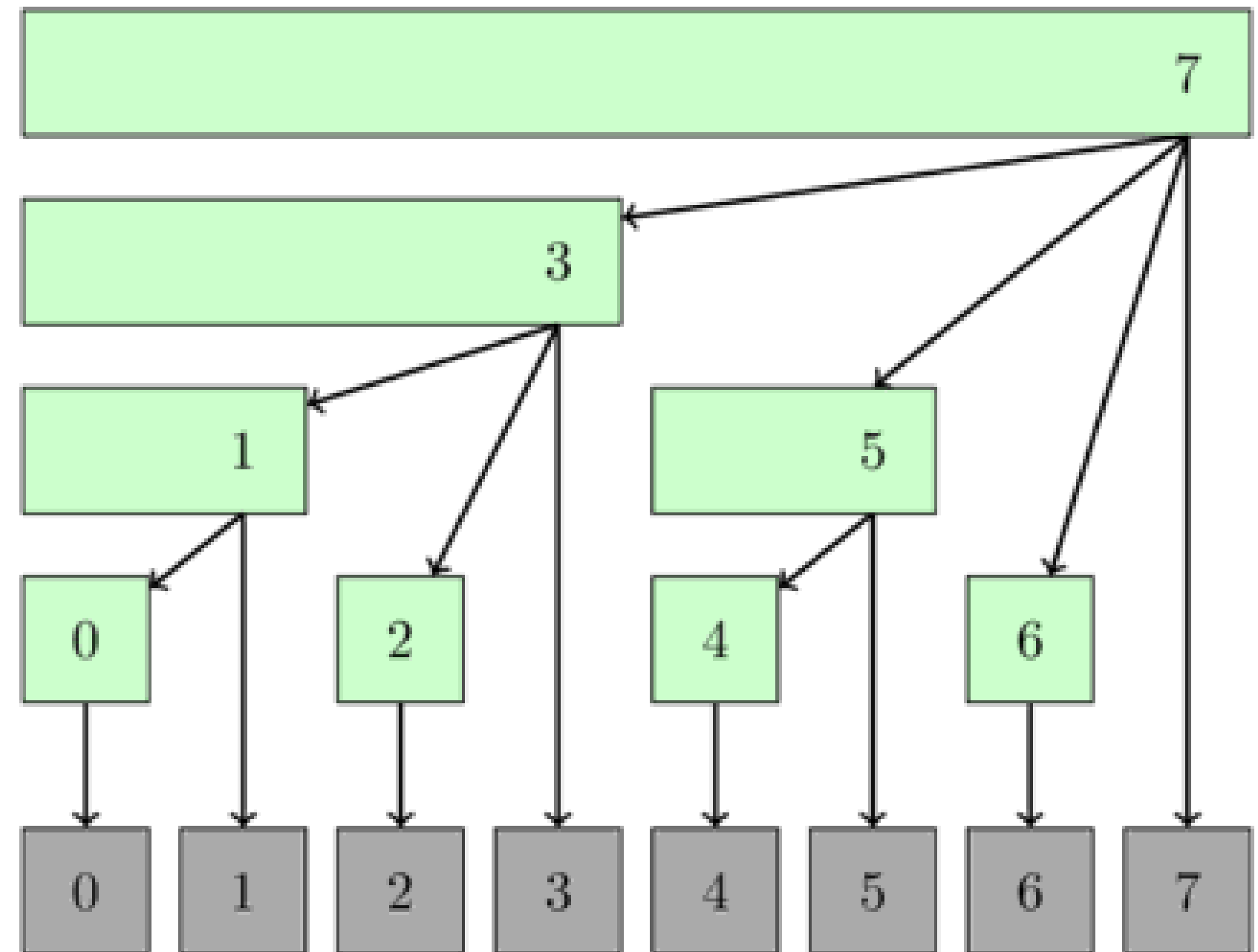
$$h(10) = 11 = 0001011_2$$

$$h(11) = 15 = 0001111_2$$

$$h(15) = 31 = 0011111_2$$

$$h(31) = 63 = 0111111_2$$

⋮



Construção Linear

O número i -ésima posição $A[i]$ contribui para o intervalo armazenado em $T[i]$, $T[i \mid i + 1]$...

Basta que esses valores sejam **propagados** ao ler $A[i]$

Código

```
struct BIT {
    int n;
    vector<ll> t;

    BIT(int n) : n(n) {
        t.assign(n, 0);
    }
    BIT(vector<ll> const &a) {
        n = a.size();
        t.assign(n, 0);
        for (int i = 0; i < a.size(); ++i) {
            t[i] += a[i];
            int r = i | (i + 1);
            if (r < n) t[r] += t[i];
        }
    }
};
```

```
ll query(int r) {
    ll ret = 0;
    for (; r >= 0; r = (r & (r + 1)) - 1) ret += t[r];
    return ret;
}

ll query(int l, int r) {
    return query(r) - query(l - 1);
}

void update(int i, ll add) {
    for (; i < n; i = i | (i + 1)) t[i] += add;
}
```

One-based indexing

Existe uma mais **elegante** (e **simples**!) de definir a BIT com pequenas mudanças em **$T[i]$** e na função **g**

Para isso, **$T[i]$** guarda a aplicação de **f** ao longo do intervalo **$[g(i) + 1, i]$**

```

1  int query(int r) {
2      int ret = 0;
3      while (r > 0) {
4          ret += t[r];
5          r = g(r);
6      }
7      return ret;
8  }
9
10 void update(int i, int add) {
11     for (int j = 0; j < N; j++) {
12         if (g(j) < i <= j) {
13             t[j] += add;
14         }
15     }
16 }

```

As únicas mudanças
foram no while com $r > 0$
e $r = g(r)$

Além disso, $g(i) < i \leq j$

Nova função g

$g(i)$ apaga o primeiro bit ligado de i

$$g(7) = g(111_2) = 110_2 = 6$$

$$g(6) = g(110_2) = 100_2 = 4$$

$$g(4) = g(100_2) = 000_2 = 0$$

Dessa forma, $g(i) = i - (i \& (-i))$

O caminho inverso para o update pode ser feito com $h(i) = i + (i \& (-i))$

Código

```
struct BIT {
    int n;
    vector<ll> t;

    BIT(int n) : n(n) {
        t.assign(n + 1, 0);
    }
    BIT(vector<ll> const &a) {
        n = a.size();
        t.assign(n + 1, 0);
        for (int i = 1; i <= n; ++i) {
            t[i] += a[i - 1];
            int r = i + (i & -i);
            if (r <= n) t[r] += t[i];
        }
    }
};
```

```
ll query(int r) {
    ll ret = 0;
    for (++r; r > 0; r -= r & -r) ret += t[r];
    return ret;
}

ll query(int l, int r) {
    return query(r) - query(l - 1);
}

void update(int i, ll add) {
    for (++i; i <= n; i += i & -i) t[i] += add;
}

};
```

Range Update and Point Query

É possível fazer updates em um intervalo em **$O(\log N)$** utilizando uma técnica de **array de diferenças**. Contudo, com essa estrutura, só é possível responder queries pontuais

$A = [3, 5, 2, 8, 1] \rightarrow D = [3, 2, -3, 6, -7]$

Após **$\text{update}(1, 3, 10)$**

$A = [3, 15, 12, 18, 1] \rightarrow D = [3, 12, -3, 6, -17]$

Código

```
struct BIT {
    int n;
    vector<ll> t;

    BIT(int n) : n(n) {
        t.assign(n + 1, 0);
    }
    BIT(vector<ll> const &a) {
        n = a.size();
        t.assign(n + 1, 0);
        for (int i = 1; i <= n; ++i) {
            t[i] += a[i-1] - (i > 1 ? a[i-2] : 0);
            int j = i + (i & -i);
            if (j <= n) t[j] += t[i];
        }
    }

    ll query(int i) {
        ll ret = 0;
        for (++i; i > 0; i -= i & -i) ret += t[i];
        return ret;
    }

    void update(int i, ll add) {
        for (++i; i <= n; i += i & -i) t[i] += add;
    }
    void update(int l, int r, ll add) {
        update(l, add);
        update(r + 1, -add);
    }
};
```

Extra:

- **Range Update and Range Query**
- **2D BIT**

REFERÊNCIAS

- Biblioteca de Algoritmos
- Fenwick Tree — CP-Algorithms
- Binary Indexed Trees — TopCoder
- Point Update Range Sum — USACO Guide
- Grupo Abeliano — Teoria de Grupos (UFRJ)